
Why Language Models Transfer to Time Series

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 TODO

2 **1 Introduction**

3 **2 Related work**

4 **Time-series foundation models (TSFMs).** Models designed as general-purpose time series fore-
5 casters, trained on Large-scale multi-domain data

6 1. Chronos []

7 2. Lag-Llama []

8 3. MOMENT []

9 4. TimeGPT []

10 **LLM-to-time-series transfer.** Work that repurposes pretrained language models for time series,
11 primary related work

12 1. "Are Language Models Actually Useful for Time Series Forecasting?" [?]

13 2. Time-LLM: Time Series Forecasting by Reprogramming Large Language Models [?]

14 3. Using Pre-trained LLMs for Multivariate Time Series Forecasting [?]

15 4. isionTS: Visual Masked Autoencoders Are Free-Lunch Zero-Shot Time Series Forecasters
16 [?]

17 **Time Series Tokenization and Representation; LLM2TS continued**

18 1. A Time Series is Worth 64 Words [?]

19 2. Small Vocabularies, Big Gains [?]

20 **The Platonic Representation Hypothesis? Cross-Modal Geometry?** Theoretical/empirical work
21 on representation convergence to explain why might representations transfer across modalities ->
22 Language to TS

23 1. The Platonic Representation Hypothesis [?]

24 2. Harnessing the Universal Geometry of Embeddings [?]

25 3. Training Large Language Models to Reason in a Continuous Latent Space [?]

26 **Scaling Laws and Efficient Adaptation** How (do we) do models scale and fine-tuning strategies
 27 interact with transfer

28 1. Scaling Laws for Transfer [?]

29 2. Studies on when scaling hurts? []

30 3. Parameter-efficient fine-tuning comparisons, LoRA vs Full vs Adapters []

31 The findings that pretrained 0.6B matches random 4B implies a transfer multiplier of $\sim 8\times$ in effective
 32 parameters; io_only regime (embedding + lm_head only) indicate if the transfer is fully captured by
 33 the i/o layers $\rightarrow$ PEFT/LoRA

34 **3 Establishing Transfer Learning is Real**

35 **Setup.** All models share the Qwen3 architecture. We train pairs of **FT** (pretrained $\rightarrow$ finetuned on
 36 TS) and **RI** (random init $\rightarrow$ trained on TS) across multiple scales (0.6B, 1.7B, 4B) using the same
 37 TS training procedure (uniform binning, 512 bins, context length 512). All models are evaluated on
 38 GiftEval (46 datasets).

39 **Main result.** FT consistently outperforms RI across scales and datasets.

Table 1: ND[0.5] on GiftEval (lower is better). FT outperforms RI at all scales.

Model	0.6B	1.7B	4B
FT (pretrained + TS)	TODO	TODO	TODO
RI (random + TS)	TODO	TODO	TODO
Δ (FT – RI)	TODO	TODO	TODO

40 **Scaling interaction.** Pretrained 0.6B matches or exceeds random-init 4B, implying a transfer
 41 multiplier of $\sim 8\times$ in effective parameters.

42 **Finetuning regime comparison.** We compare full finetuning, LoRA, and IO-only (embedding +
 43 LM head only).

44 **4 The Pretrained Manifold: An Affinity for Transfer**

45 **4.1 A Linear Probe Reveals Latent Temporal Structure**

46 We ask: before any finetuning on time series, do pretrained language model representations already
 47 contain temporal structure? To test this directly, we train a single linear projection from a frozen
 48 pretrained model’s hidden states to produce time series—with no paired text–time-series supervision
 49 and no modification to the model weights.

50 **Method.** For each WikiText sequence, we extract hidden states from all 28 layers and concatenate
 51 them per timestep: $\mathbf{h}_{i,t} = [\mathbf{h}_{i,t}^{(0)}; \dots; \mathbf{h}_{i,t}^{(27)}] \in \mathbb{R}^D$, $D = 28,672$. A per-timestep linear map
 52 $\hat{\mathbf{y}}_{i,t} = \mathbf{w}^\top \mathbf{h}_{i,t} + b$ produces a predicted sequence $\hat{\mathbf{y}}_i \in \mathbb{R}^T$, z-normalized to zero mean and unit
 53 variance. Training uses EM-style nearest-neighbor matching: each prediction is matched to its closest
 54 real time series from GiftEval, and the map is optimized to minimize MSE to these assignments, with
 55 a spectral diversity penalty to prevent collapse.

56 **Result.** The linear map produces 686 unique matches out of 10,000 real time series, spanning
 57 14 of 20 temporal clusters. The map generalizes to held-out text (439 unique on unseen WikiText
 58 sequences). This is only possible if the pretrained representation space already contains directions
 59 whose projections resemble real temporal signals.

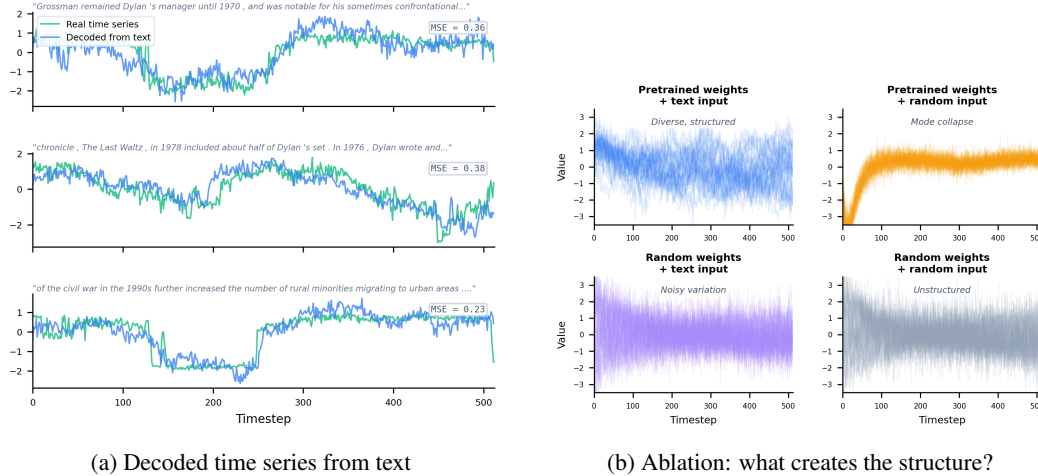


Figure 1: **Pretrained hidden states contain time-series-compatible structure.** A single linear map $\hat{y}_t = \mathbf{w}^\top \mathbf{h}_t + b$ is trained on frozen LLM hidden states via EM-style nearest-neighbor matching to a bank of 10,000 real time series (no paired data). **(a)** Three decoded outputs (blue) overlaid with their nearest real time series (green). Input text shown above each plot. $\text{MSE} = 0.23\text{--}0.38$; full distribution in Appendix. **(b)** Ablation crossing learned weights vs. random init with text vs. random input. *Top-left*: pretrained + text—diverse, structured outputs (301 unique matches). *Top-right*: pretrained + random tokens—mode collapse to one shape (manifold exists but is not traversed). *Bottom-left*: random weights + text—noisy variation without structure. *Bottom-right*: random weights + random tokens—unstructured. Only pretrained weights *and* meaningful text together produce diverse temporal signals.

What creates this structure? The result above could arise from multiple sources: the transformer architecture itself (residual connections produce smooth trajectories), the learned weights (pretraining compresses representations), or the input (natural text has sequential structure). We disentangle these by crossing model weights (pretrained vs. randomly initialized, same architecture) with input (WikiText vs. random tokens):

Table 2: 2×2 ablation. *Unique*: distinct real TS matched. *NN*: mean nearest-neighbor MSE. Low NN with low Unique indicates mode collapse, not quality.

Condition	Train Unique	Held-Out Unique	Held-Out NN	Clusters
text + PT	686/1920	439/1000	0.717	14/20
text + RandInit	54/1920	73/1000	0.541	13/20
rand + PT	4/1920	1/1000	0.293	4/20
rand + RandInit	99/1920	138/1000	0.741	14/20

The ablation reveals a clear interaction. **Pretrained weights** create a structured, low-dimensional manifold—but random tokens fail to traverse it (rand+PT collapses to 4 unique outputs). **Meaningful text** provides variation—but without pretrained weights, this variation is unstructured noise (text+RandInit). Only the combination produces diverse, high-quality temporal signals: the pretrained manifold provides the geometry, and text input drives diverse traversal of it.

4.2 The Projected Dynamics Forecast Beyond the Context

The linear decoding result shows shape-level similarity between projected hidden states and real time series. We test whether this extends to *predictive* structure: given the first half of a time series, can a retrieved text projection forecast the second half?

Method. We take 500 real time series from GiftEval and observe only the first 256 of 512 timesteps. Among 10,000 WikiText projections, we retrieve the one with lowest MSE on the observed first half.

We then use that projection’s second half as the forecast. The baseline is last-value carry-forward (repeating the final observed value).

Result. Across all 500 queries, the retrieval-based forecast achieves $\text{MSE} = 1.91$ versus last-value’s 2.27 (16% lower). The retrieval wins in 37% of cases; when it wins, the advantage is large (median $4\times$ lower MSE), because the retrieved projection captures trends and level shifts that last-value misses entirely. Figure 2 shows the best-case example from our evaluation set.

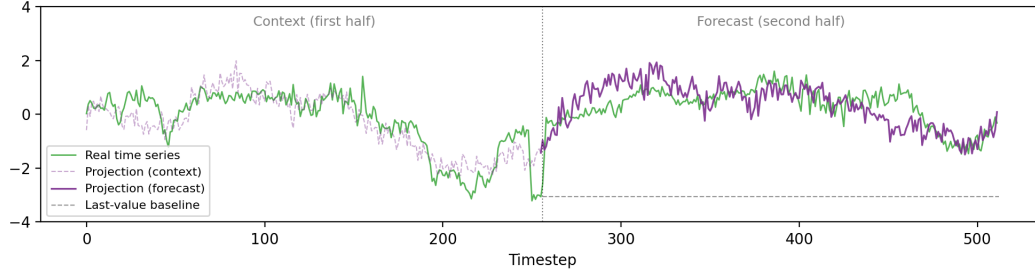


Figure 2: Best-case forecasting example from 500 evaluation queries. The first 256 timesteps are used to retrieve the closest WikiText projection; the projection’s second half (purple) forecasts the continuation. The retrieved projection captures the level shift and subsequent trend ($\text{MSE} = 0.42$), while last-value carry-forward (gray dashed) is stranded at the final context value ($\text{MSE} = 11.7$).

4.3 Why the Manifold Enables Efficient Transfer

The linear decoding result has a direct implication for finetuning. Let $\mathcal{M} \subset \mathbb{R}^D$ denote the manifold of hidden-state trajectories produced by the pretrained model processing text. The linear decoding experiment shows that there exists a direction $\mathbf{w} \in \mathbb{R}^D$ such that the distribution of projected trajectories $\{\mathbf{w}^\top \mathbf{h}_t\}_{t=1}^T$ has significant overlap with the distribution of real time series P_{TS} .

If projections of $\mathcal{M}$ already approximate P_{TS} , then adapting the model to predict time-series tokens autoregressively requires only identifying which directions in $\mathcal{M}$ are useful for temporal prediction and amplifying them. This is a low-rank transformation of the existing representation, rather than learning temporal dynamics from scratch across the full dimensionality of the space.

4.4 Pretraining Creates Coherent Gradient Signal for the TS Objective

The manifold structure has a direct consequence for optimization. We compute per-example gradients of the TS loss at the pretrained versus random initialization, measuring whether individual training examples provide coherent or contradictory optimization signal.

Method. For 30 individual TS sequences, we compute the gradient $\mathbf{g}_i = \nabla_{\theta} L_i$ restricted to layers 6–10 (78.7M parameters) at both initializations. We measure pairwise cosine similarity $\cos(\mathbf{g}_i, \mathbf{g}_j)$ across all example pairs, and the signal-to-noise ratio $\text{SNR} = \|\bar{\mathbf{g}}\| / \text{mean}(\|\mathbf{g}_i - \bar{\mathbf{g}}\|)$.

Table 3: Per-example gradient structure at the TS loss surface. At PT initialization, different TS examples produce gradients that reinforce each other. At random initialization, gradients are nearly orthogonal and cancel upon averaging.

Metric	PT init	Random init
Per-example gradient cosine	0.58	0.04
Gradient signal-to-noise ratio	1.16	0.28
Gradient norm (RMS)	0.0040	0.0004

Interpretation. At PT initialization, per-example TS gradients have cosine similarity 0.58—different time series sequences agree on which direction to adjust the weights. Batch averaging reinforces this shared signal. At random initialization, gradients are nearly orthogonal (cosine 0.04,

consistent with random vectors in $\sim 78\text{M}$ dimensions), so averaging causes massive cancellation: the SNR drops to 0.28, meaning most of the per-example gradient signal is lost.

This coherence arises because the pretrained representations create consistent feature activations across different TS inputs. When the loss asks “how should this weight change?”, the answer is similar regardless of which TS sequence is used, because the pretrained features impose a shared structure on the loss landscape. At random initialization, each example activates arbitrary features, giving contradictory signals.

The effect is layer-dependent: gradient norms at PT initialization are $3\text{--}19\times$ larger than at random initialization, with the ratio increasing for deeper layers (where random weights scramble the signal most). This layer-wise gradient structure explains why mid-to-deep layers benefit most from pretrained initialization (Section 5.2).

Curvature anisotropy. To characterize the *shape* of the loss landscape, we compute the curvature $\mathbf{v}^\top H \mathbf{v}$ along 200 random unit vectors $\mathbf{v}$ in layers 6–10 parameter space using exact Hessian-vector products (Pearlmutter trick), and compare this to the curvature along the gradient direction $\hat{\mathbf{g}}$. At PT initialization, the gradient direction has curvature 232, while the mean curvature across random directions is 9.6×10^{-5} —a ratio of 2.4×10^6 . At random initialization, the gradient direction has curvature -0.08 (near-zero, slightly negative), barely distinguishable from random directions (mean 8.4×10^{-7} , ratio $\sim 80\times$). The PT landscape is thus *massively anisotropic*: SGD’s natural descent direction is aligned with one of the very few high-curvature directions in a $\sim 78\text{M}$ -dimensional space, while the vast majority of directions are flat. At random initialization, the landscape is nearly isotropic—all directions, including the gradient, have negligible curvature.

5 Finetuning Projects the Pretrained Manifold onto Time Series

5.1 The Weight Changes are Low-Rank

Table 4: Finetuning regime comparison on GiftEval ND[0.5]. TODO: fill from Alexis’s experiments.

Method	Params changed	ND[0.5]	% of full FT
Full finetuning	100%	TODO	100%
LoRA ($r = 16$)	TODO%	TODO	TODO%
LoRA ($r = 4$)	TODO%	TODO	TODO%
IO-only (emb + head)	TODO%	TODO	TODO%
Random init (full)	100%	TODO	—

Interpretation. Only a low-rank perturbation of the pretrained weights is needed to adapt the model from language to time series. This means the bulk of the pretrained computation is preserved—finetuning adjusts a small subspace of directions.

5.2 Finetuning Selects Useful Directions

Catastrophic forgetting as geometric compression. Text $\rightarrow$ FT collapses to effective rank 2.5 (from 150 in PT). Finetuning traded ~ 140 text-useful dimensions for ~ 12 TS-useful dimensions.

5.3 Reuse vs. Reinvention: FT and RI Find Different Solutions

To examine how each model handles different temporal structures, we pass five synthetic inputs through all three models and visualize the hidden-state trajectories at Layer 8 via 2D PCA (Figure 5). Each input is normalized and tokenized identically using uniform binning. We fit PCA independently per model–input pair, as the models occupy different regions of representation space. The first 5 positions are excluded to avoid the attention-sink artifact common in transformer first-position hidden states. Trajectories are colored by input phase (period 64) to reveal periodic structure.

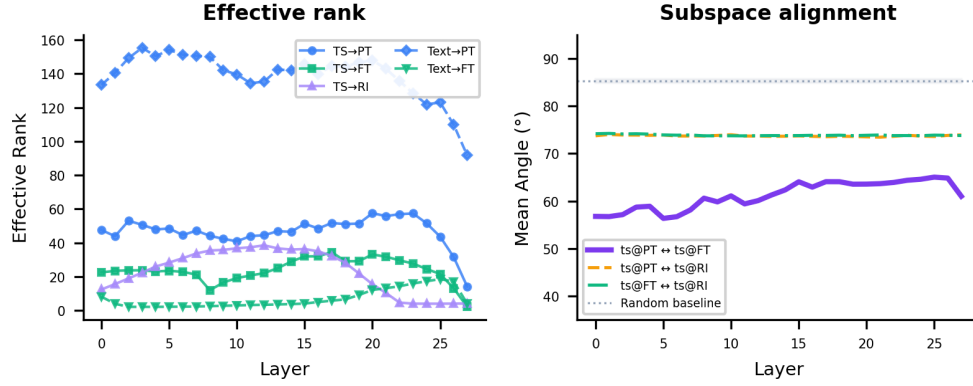


Figure 3: **Geometric evidence for direction reuse.** (Left) Effective rank across layers. FT compresses TS representations to ~ 12 dimensions (green solid), far below PT’s ~ 44 (blue solid) and RI’s ~ 35 (purple solid). Text→FT collapses to ~ 2.5 (catastrophic forgetting). (Right) Subspace alignment: mean principal angle between top-10 PCs. $ts@PT \leftrightarrow ts@FT$ (purple) stays well below the random baseline (85.3°), confirming FT reuses PT’s directions. Tables 6 and 7 in Appendix.

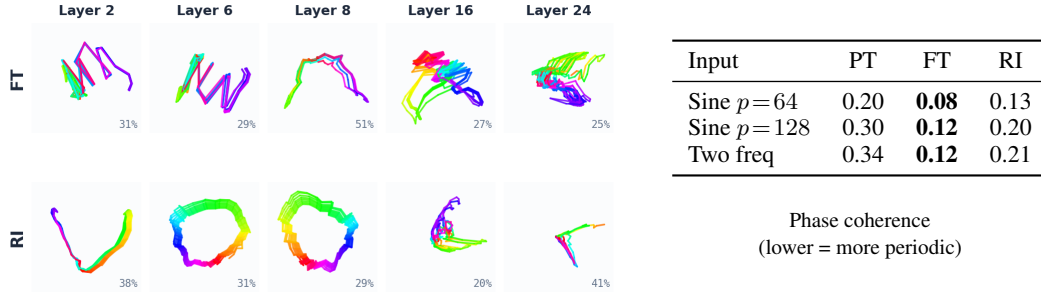


Figure 4: **Reuse vs. reinvention.** (Left) A sine wave (period 64) through FT (top) and RI (bottom) across layers, shown as 2D PCA of hidden-state trajectories colored by input phase. Percentages show variance explained by 2 PCs. FT’s loops are geometrically complex (inherited from pretraining) while RI discovers clean, symmetric circles from scratch. At Layer 8, FT captures 51% of variance in 2D vs. RI’s 29%—FT concentrates periodic structure into fewer dimensions. (Right) Phase coherence (ratio of same-phase to all-pairs distance, computed in the full 1024-dim hidden space; lower = more periodic). FT represents periodicity most explicitly at all tested inputs.

5.4 Interpreting the Transferred Features

Method. A crosscoder (shared encoder, per-domain decoders, TopK=64, 4096 latent dims) is trained on PT/FT/RI activations at each layer. Features are categorized by which domains activate them. PT-involving features are probed on WikiText-103.

Cross-domain features. The top PT_FT features encode *structural temporal primitives* shared between text and time series:

Interpretation. The transferred features detect *computational patterns*—repetition, gradients, periodicity—not semantic content. These are primitives useful for any sequential prediction task. Language pretraining learns them because text also requires tracking repetition, change, and rhythm.

6 Discussion and Limitations

The full picture.

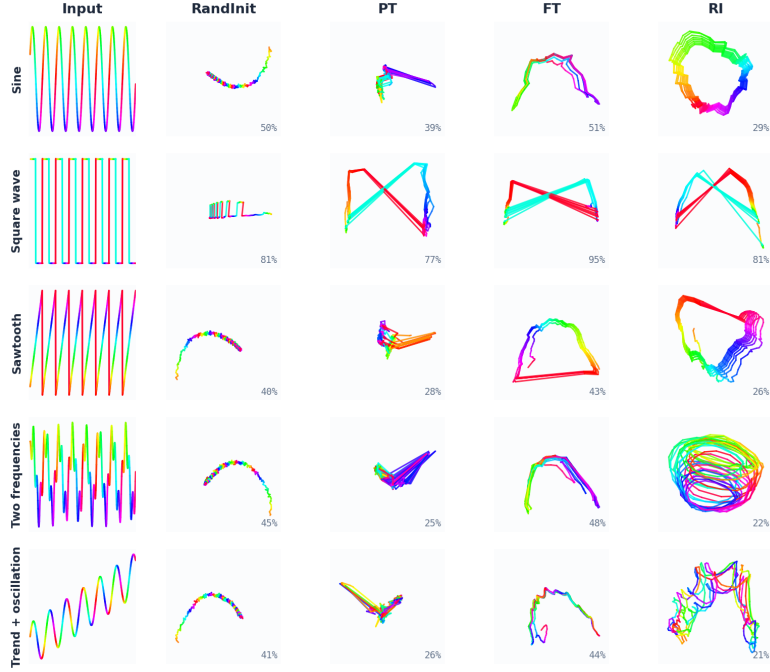


Figure 5: **Hidden-state trajectories for synthetic inputs at Layer 8.** Each row shows a different input waveform (left) and its 2D PCA trajectory through PT, FT, and RI (colored by phase). Percentages show variance captured by 2 PCs. PT produces thin, low-variance trajectories (it was not trained on these tokens). FT creates structured, high-variance loops—especially for periodic inputs (sine: 51%, square: 95%). RI produces geometrically distinct solutions: clean circles for periodic inputs but using more total dimensions. The two-frequency input reveals the starkest contrast: FT produces a compact trajectory while RI forms a complex multi-loop orbit.

Table 5: Top cross-domain features (PT_FT). Each fires on specific text patterns *and* time series patterns.

Layer	Name	Text activation	TS activation
8	Repetition detector	“3 · 3 · 3 · 3”, “2–3–2 config”	Square waves, periodic oscillations
26	Slope detector	“derivative”, “tangent line”	Rising/falling slopes
4	Metronome detector	“tempo”, “beats”	Sharp periodic spikes
5	Chord detector	“F–E $\flat$ –G” progressions	Periodic peaks

148 **Implications.**

149 **Limitations.**

150 **7 Conclusion**

Table 6: Representational geometry at Layer 8 for three models processing the same TS input.

Condition	Eff. Rank	PCs for 90%	Speed	Low Freq %
TS $\rightarrow$ PT	44 ± 29	63	16	34.5%
TS $\rightarrow$ FT	12 ± 6	15	64	52.3%
TS $\rightarrow$ RI	35 ± 21	46	78	44.7%
Text $\rightarrow$ PT	150 ± 8	166	24	21.4%
Text $\rightarrow$ FT	2.5 ± 1.5	3	355	16.4%

Table 7: Mean principal angle between top-10 PCs of each condition pair at Layer 8. Random baseline: $85.3^\circ \pm 0.4^\circ$.

Pair	Mean Angle	Significance
ts@PT vs ts@FT	60.6°	most aligned
ts@PT vs ts@RI	73.7°	aligned but less
ts@FT vs ts@RI	73.7°	—
text@PT vs text@FT	79.9°	partially shifted
text@PT vs text@RI	85.4°	$\approx$ random

Table 8: Top cross-domain features (PT_FT). Each fires on specific text patterns *and* time series patterns.

Layer	Name	Text activation	TS activation
8	Repetition detector	“3 · 3 · 3 · 3”, “2–3–2 config”	Square waves, periodic oscillations
26	Slope detector	“derivative”, “tangent line”	Rising/falling slopes
4	Metronome detector	“tempo”, “beats”	Sharp periodic spikes
5	Chord detector	“F–Eb–G” progressions	Periodic peaks

